



SmartSus

Sistema Inteligente de Gestão da Fila Cirúrgica do SUS

**Documentação Técnica — Modelagem Matemática e
Pesquisa Operacional Aplicada à Saúde Pública**

Instituição: FECAF — Fundação Educacional do Vale do Jaguari
Disciplina: Operations Research
Professor: Jesse Fernandes
Aluna: Isabela Costa
Versão: 1.0.0 — Maio 2026

1. Introdução

O **SmartSus** é um sistema computacional desenvolvido para otimizar a gestão da fila de espera de cirurgias eletivas no Sistema Único de Saúde (SUS) do estado de São Paulo. O sistema aplica conceitos formais de **Pesquisa Operacional (PO)** para substituir a lógica tradicional de atendimento por ordem de chegada (FIFO) por um modelo matemático de priorização multicritério, garantindo que os casos mais graves sejam atendidos primeiro dentro dos limites temporais legais.

O sistema integra o **Problema de Transporte (TP)** e o **Problema de Designação (AP)**, dois modelos clássicos da PO formalizados por Hitchcock (1941) e Kuhn (1955), adaptados ao contexto hospitalar com 10 unidades reais da Grande São Paulo, cada uma com capacidade de 20 cirurgias por dia.

1.1 Objetivos do Sistema

- Otimizar a fila de espera de pacientes cirúrgicos do SUS
- Priorizar pacientes com base em critérios médicos e sociais objetivos
- Encaminhar automaticamente cada paciente ao hospital mais próximo com disponibilidade
- Garantir que nenhum paciente aguarde mais de 180 dias (restrição legal)
- Minimizar o tempo total ponderado de espera da fila

1.2 Tecnologias Utilizadas

Componente	Tecnologia	Função
Backend	Python + FastAPI	API REST e lógica de negócio
Banco de Dados	MySQL	Persistência de dados
Solver PO	PuLP + CBC	Resolução do modelo matemático
Roteamento	OpenRouteService API	Cálculo de distância e tempo real
Frontend	React + Vite + TailwindCSS	Interface do administrador

2. Fundamentação Teórica — Pesquisa Operacional

A Pesquisa Operacional utiliza matemática, lógica, estatística e computação para auxiliar na tomada de decisões em problemas complexos. O principal objetivo é encontrar a melhor solução possível utilizando os recursos da forma mais eficiente. O SmartSus aplica dois modelos clássicos da PO:

2.1 Problema de Transporte (TP)

Formalizado por **Frank L. Hitchcock (1941)** e expandido por **Tjalling Koopmans** (Prêmio Nobel de Economia, 1975), o Problema de Transporte trata do envio de mercadorias de origens para destinos ao menor custo possível.

Aplicação no SmartSus: As *origens* são os endereços dos pacientes (bairros de SP) e os *destinos* são os 10 hospitais parceiros. O custo c_{ij} representa a distância em km e tempo de deslocamento calculados pela API OpenRouteService, com fator de correção 1.8x para o trânsito real de São Paulo.

Modelo Matemático do TP:

$$\text{Minimizar } Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} \cdot x_{ij}$$

Elemento	Descrição	No SmartSus
Origens (i)	Pontos de partida	Endereços dos pacientes
Destinos (j)	Pontos de chegada	10 hospitais de SP
Custo c_{ij}	Custo de transporte	Distância km + tempo (ORS)
Oferta s_i	Disponibilidade na origem	1 paciente por cirurgia
Demanda d_j	Capacidade no destino	20 cirurgias/dia/hospital

2.2 Problema de Designação (AP)

Formalizado por **Harold Kuhn (1955)** com o Método Húngaro, o Problema de Designação aloca recursos (designados) a tarefas de forma ótima. Cada recurso é atribuído a exatamente uma tarefa, minimizando o custo total.

Aplicação no SmartSus: Os *designados* são os pacientes e as *tarefas* são os slots de cirurgia (hospital + data). A variável de decisão $x_{ij} \in \{0,1\}$ indica se o paciente i é alocado ao slot j . O custo c_{ij} é o inverso do score de prioridade, forçando o solver a alocar primeiro os pacientes mais urgentes.

Modelo Matemático do AP:

$$\text{Minimizar } Z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot x_{ij}$$

3. Variáveis de Decisão

O modelo do SmartSus utiliza duas classes de variáveis de decisão que determinam o comportamento do sistema de otimização:

3.1 Variável de Alocação (Binária)

$$x_{ij} \in \{0, 1\}$$

Onde *i* representa o índice do paciente (*i* = 1, 2, ..., *n*) e *j* o índice do slot de cirurgia (hospital + data). A variável assume valor **1** quando o paciente *i* é alocado ao slot *j*, e **0** caso contrário.

x_{ij} = 1	Paciente <i>i</i> alocado ao hospital <i>j</i> na data <i>d</i>	Cirurgia agendada
x_{ij} = 0	Paciente <i>i</i> não alocado ao slot <i>j</i>	Slot não utilizado

3.2 Variável de Score de Prioridade (Contínua)

$$S_i \in \mathbb{R}^+$$

O score *S_i* é uma variável contínua não-negativa calculada para cada paciente por uma função multicritério ponderada. Determina a ordem de prioridade na fila — quanto maior o score, maior a urgência e menor o custo *c_{ij}* atribuído ao paciente. O score é recalculado automaticamente a cada execução do algoritmo de otimização.

Índices do modelo: *i* = 1, 2, ..., *n* — pacientes na fila | *j* = 1, 2, ..., 10 — hospitais parceiros | *d* = hoje, hoje+1, ..., hoje+60 — horizonte de agendamento (60 dias)

4. Função Objetivo

A função objetivo do SmartSus busca **minimizar o tempo total ponderado de espera** de todos os pacientes na fila, dando prioridade aos casos mais graves:

$$\text{Min } Z = \sum_i \sum_j c_{ij} \cdot x_{ij}$$

Onde o custo de alocação c_{ij} é definido como:

$$c_{ij} = (1 / S_i) \times (1 + j \times 0.05)$$

Essa formulação garante que pacientes com **maior score** (mais urgentes) recebam **menor custo**, sendo priorizados pelo solver. O fator $(1 + j \times 0.05)$ penaliza slots mais distantes no tempo, incentivando agendamentos nas datas mais próximas.

Interpretação prática: O solver PuLP minimiza Z escolhendo as combinações (paciente, slot) que resultam no menor custo total. Como $c_{ij} = 1/S_i$, alocar um paciente P1 (score alto) a um slot cedo gera custo baixo — exatamente o comportamento desejado.

5. Restrições do Modelo

As restrições garantem que a solução encontrada pelo PuLP seja **viável**, respeitando os limites físicos, médicos e legais do sistema:

Restrição R1 — Unicidade de Alocação

$$\sum_{j=1}^J x_{ij} = 1 \text{ para todo paciente } i$$

Cada paciente deve ser alocado a **exatamente um slot** de cirurgia. Não é possível dividir um paciente entre hospitais ou datas distintas.

Restrição R2 — Capacidade Diária Hospitalar

$$\sum_{i=1}^n x_{ij} \leq 20 \text{ para todo hospital } j \text{ e data } d$$

Cada hospital realiza no máximo **20 cirurgias por dia**. A restrição é avaliada individualmente por slot (hospital + data), distribuindo os agendamentos ao longo do horizonte de 60 dias.

Restrição R3 — Limite Temporal Crítico

$$T_i \leq 180 \text{ dias para todo paciente } i$$

Restrição mais crítica do modelo. Nenhum paciente pode aguardar mais de 180 dias na fila — limite estabelecido pela legislação do SUS. O sistema monitora continuamente e aplica um acelerador exponencial no score de pacientes acima de 150 dias, garantindo que sejam priorizados antes do vencimento.

Restrição R4 — Não-Negatividade e Binariedade

$$x_{ij} \geq 0 \text{ e } x_{ij} \in \{0, 1\} \text{ para todo } i, j$$

As variáveis de decisão são **binárias** — um paciente ou está alocado a um slot (1) ou não está (0). Não existem alocações fracionadas.

5.1 Resumo das Restrições

Restrição	Expressão	Tipo	Quantidade
R1 — Unicidade	$\sum_j x_{ij} = 1$	Igualdade	n pacientes
R2 — Capacidade	$\sum_i x_{ij} \leq 20$	Desigualdade	10 hosp. × 60 dias = 600
R3 — 180 dias	$T_i \leq 180$	Desigualdade	n pacientes
R4 — Binariedade	$x_{ij} \in \{0,1\}$	Domínio	$n \times J$ variáveis

6. Função de Score — Priorização Multicritério

O score S_i de cada paciente é calculado por uma **combinação linear ponderada** de quatro critérios médicos e sociais. Os pesos foram definidos com base em diretrizes clínicas do SUS e literatura de triagem hospitalar:

$$S_i = \alpha \cdot M_i + \beta \cdot I_i + \gamma \cdot D_i + \delta \cdot C_i$$

com $\alpha + \beta + \gamma + \delta = 0.45 + 0.20 + 0.25 + 0.10 = 1.00$

6.1 Critério M — Gravidade Manchester (alpha = 0.45)

Recebe o **maior peso (45%)** pois representa risco imediato de vida. A Escala de Manchester é o protocolo oficial de triagem hospitalar no Brasil.

Nível	Classificação	Pontuação M_i	Justificativa
P1	Imediato	100	Risco de vida imediato — minutos
P2	Muito Urgente	80	Deterioração rápida — < 15 min
P3	Urgente	60	Estado estável mas necessita atenção
P4	Pouco Urgente	40	Pode aguardar — < 2 horas
P5	Não Urgente	20	Condição não urgente

6.2 Critério I — Fator Etário (beta = 0.20)

Peso de 20% — Critério social que prioriza idosos:

$I_i = 0$ se idade < 40 anos

$I_i = (\text{idade} - 40) \times 0.5$ se $40 \leq \text{idade} < 60 \Rightarrow [0 \text{ a } 10 \text{ pts}]$

$I_i = 10 + (\text{idade} - 60) \times 0.5$ se $\text{idade} \geq 60 \Rightarrow [10 \text{ a } 20 \text{ pts}]$

Justificativa: Idosos têm menor capacidade de recuperação espontânea e maior risco de complicações com o tempo de espera. O critério é progressivo com aceleração acima dos 60 anos.

6.3 Critério D — Tempo na Fila (gamma = 0.25)

Peso de 25% — Mecanismo anti-starvation:

$D_i = 30 \times (\text{dias} / 180)$ se $\text{dias} \leq 120$ [crescimento linear]

$D_i = 30 \times (\text{dias} / 180)^2$ se $\text{dias} > 120$ [crescimento exponencial]

Justificativa: Garante justiça na fila — evita que um paciente fique eternamente na mesma posição (problema de starvation em filas de prioridade). O crescimento exponencial após 120 dias força a subida do paciente antes de atingir o limite de 180 dias.

6.4 Critério C — Tipo de Cirurgia (delta = 0.10)

Peso de 10% — Urgência clínica por tipo de procedimento:

Cirurgia	Pts	Cirurgia	Pts	Cirurgia	Pts
Cardíaca	10	Neurológica	10	Oncológica	9
Vascular	8	Gastrointestinal	7	Ortopédica	7
Urológica	6	Ginecológica	6	Oftalmológica	6
Geral	5				

7. Implementação com PuLP

O SmartSus utiliza a biblioteca **PuLP** (Python Linear Programming) com o solver **CBC (Coin-or Branch and Cut)** para resolver o Problema de Designação. O PuLP permite formular o modelo matemático diretamente em Python e delegar a solução ao solver de forma transparente.

7.1 Formulação do Problema

O código Python que implementa o modelo:

```
prob = pulp.LpProblem('SmartSus_Designacao', pulp.LpMinimize)
# Variáveis binárias x[i][j]
x[(i,j)] = pulp.LpVariable(f'x_{i}_{j}', cat=pulp.LpBinary)
# Função objetivo: Min Z = sum c_ij * x_ij
prob += pulp.lpSum(custo(i,j) * x[(i,j)] for i,j in pares)
# R1: cada paciente em exatamente 1 slot
prob += pulp.lpSum(x[(i,j)] for j in slots) == 1
# R2: capacidade maxima por slot
prob += pulp.lpSum(x[(i,j)] for i in pacientes) <= 20
# Resolver com CBC
prob.solve(pulp.PULP_CBC_CMD(msg=0, timeLimit=30))
```

7.2 Estratégia de Lotes

Para manter performance com grandes volumes de pacientes, o algoritmo divide a fila em **lotes de 80 pacientes**, ordenados por score decrescente. Cada lote é resolvido independentemente, mas compartilha o contador de vagas em memória para evitar conflitos de capacidade entre lotes.

7.3 Saída do Solver

Campo	Tipo	Descrição
status_pulp	String	Optimal / Feasible / Infeasible
pacientes_agendados	Integer	Total de pacientes alocados
valor_objetivo	Float	Valor de Z na solução ótima
num_variaveis	Integer	Total de variáveis x_{ij} criadas
num_restricoes	Integer	Total de restrições ativas no modelo
metodo	String	PuLP CBC — Programação Linear Inteira Mista

8. Exemplo Numérico

Considere 3 pacientes aguardando cirurgia:

Paciente	Gravidade	Idade	Dias Fila	Cirurgia	Score S _i
João Silva	P1	72 anos	45 dias	Cardíaca	52.3
Maria Costa	P3	45 anos	160 dias	Ortopédica	38.7
Carlos Souza	P2	38 anos	20 dias	Neurológica	37.2

Cálculo detalhado do score de João Silva (P1, 72 anos, 45 dias, Cardíaca):

M_i = 100 pts (P1 – Imediato)

I_i = 10 + (72 - 60) x 0.5 = 10 + 6 = 16 pts

D_i = 30 x (45/180) = 7.5 pts

C_i = 10 pts (Cardiaca)

S_i = 0.45x100 + 0.20x16 + 0.25x7.5 + 0.10x10 = 45+3.2+1.875+1 = 52.3

Resultado: João (score 52.3) é alocado primeiro pelo PuLP. Maria tem score alto (38.7) por causa dos 160 dias na fila — mecanismo anti-starvation funcionando. Carlos, apesar de P2, tem poucos dias na fila e menos idade, resultando em score menor.

8.1 Fila FIFO vs Fila Otimizada

Comparação da ordem de atendimento com e sem o algoritmo:

Posição	FIFO (Chegada)	Score	Otimizado (PuLP)	Score
#1	Carlos Souza (chegou 1o)	37.2	João Silva (P1)	52.3
#2	João Silva	52.3	Maria Costa (160d)	38.7
#3	Maria Costa	38.7	Carlos Souza	37.2

Na FIFO, Carlos (P2, 20 dias) seria atendido antes de João (P1, 45 dias) — decisão clinicamente inadequada.

9. Hospitais Parceiros

O sistema integra **10 hospitais reais** da Grande São Paulo, com cobertura estratégica de todas as regiões metropolitanas:

#	Hospital	Bairro/Cidade	Região	Cap./Dia
1	Hospital das Clínicas da FMUSP	Cerqueira César	Centro	20
2	Hospital do Servidor Público Estadual	Vila Clementino	Sul	20
3	Hospital Geral de Taipas	Taipas	Zona Norte	20
4	Hospital Geral do Grajaú	Grajaú	Zona Sul	20
5	Hospital Geral de Pedreira	Pedreira	Zona Sul	20
6	Hospital Estadual de Sapopemba	Sapopemba	Zona Leste	20
7	Hospital Geral de Itaquaquetuba	Itaquaquetuba	Zona Leste	20
8	Hospital Estadual Mário Covas	Santo André	ABC	20
9	Hospital Geral de Carapicuíba	Carapicuíba	Zona Oeste	20
10	Hospital Regional de Cotia	Cotia	Zona Oeste	20

Capacidade total do sistema: 10 hospitais × 20 cirurgias/dia = **200 cirurgias/dia** · Horizonte de agendamento: 60 dias · Capacidade total projetada: **12.000 cirurgias**

10. Fluxo Completo do Sistema

Etapa	Ação	Componente
1	Cadastro manual ou simulação de pacientes	Frontend / API
2	Cálculo automático do score S_i	services/priorizacao.py
3	Exibição da fila em ordem de chegada (FIFO)	Fila.jsx — modo FIFO
4	Administrador clica 'Otimizar Fila'	Frontend
5	API recalcula scores e monta lotes de 80	routers/agendamento.py
6	PuLP formula o modelo LP com x_{ij} e restrições	services/otimizacao_pulp.py
7	CBC solver resolve: $\text{Min } Z = \sum c_{ij} * x_{ij}$	PuLP CBC
8	Alocações aplicadas no banco MySQL	SQLAlchemy / MySQL
9	Frontend exibe status Optimal + métricas PuLP	Fila.jsx
10	Administrador verifica agenda por hospital/dia	Agenda.jsx
11	Cirurgia realizada — status atualizado	Fila.jsx — botão Realizado

10.1 Referências Bibliográficas

- HITCHCOCK, F.L. *The Distribution of a Product from Several Sources to Numerous Localities*. Journal of Mathematics and Physics, 1941.
- KOOPMANS, T.C. *Optimum Utilization of the Transportation System*. Econometrica, 1949.
- KUHN, H.W. *The Hungarian Method for the Assignment Problem*. Naval Research Logistics Quarterly, 1955.
- HILLIER, F.S.; LIEBERMAN, G.J. *Introduction to Operations Research*. 10th ed. McGraw-Hill, 2015.
- MITCHELL, S. et al. *PuLP: A Linear Programming Toolkit for Python*. 2011. Disponível em: <https://coin-or.github.io/pulp/>
- MINISTÉRIO DA SAÚDE. *Portaria GM/MS nº 1.559/2008 — Regulação do acesso à assistência*. Brasília, 2008.

Contato: Sistema desenvolvido como projeto acadêmico da disciplina Operations Research · FECAF · Professor Jesse Fernandes · jesse.fernandes@pro.fecaf.com.br